Week 4 • Class 2 — Agentic RAG Decision Systems

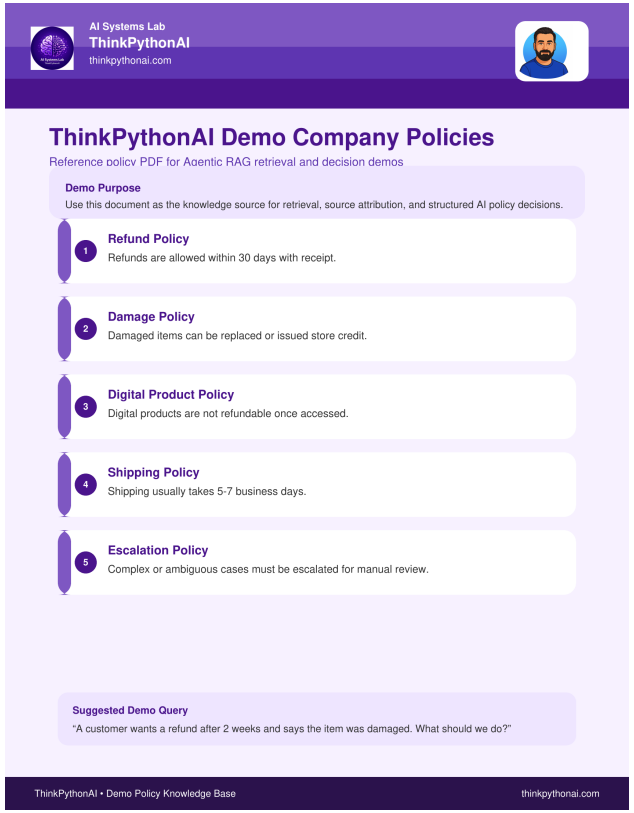
Planner + Retrieval + Decision + Structured Output

AGENTIC RAG

PDF → Planner JSON → Retrieval → Decision JSON

Combining Week 2 JSON, Week 3 RAG, and Week 4 agents into one production-style AI system.

Live demo knowledge source



AI Systems Lab
ThinkPythonAI
thinkpythonai.com

ThinkPythonAI Demo Company Policies

Reference policy PDF for Agentic RAG retrieval and decision demos

Demo Purpose
Use this document as the knowledge source for retrieval, source attribution, and structured AI policy decisions.

1. **Refund Policy**
   Refunds are allowed within 30 days with receipt.
2. **Damage Policy**
   Damaged items can be replaced or issued store credit.
3. **Digital Product Policy**
   Digital products are not refundable once accessed.
4. **Shipping Policy**
   Shipping usually takes 5-7 business days.
5. **Escalation Policy**
   Complex or ambiguous cases must be escalated for manual review.

Suggested Demo Query
"A customer wants a refund after 2 weeks and says the item was damaged. What should we do?"

ThinkPythonAI • Demo Policy Knowledge Base thinkpythonai.com

ThinkPythonAI Demo Company Policies.pdf

Continuity: Week 2 + Week 3 + Week 4

Week 2

JSON contracts
validation
guardrails

Week 3

RAG retrieval
embeddings
Top-K

Week 4

planner
agent loop
state

Today

Agentic RAG
decision system

Week 2 gave structure.
Week 3 gave knowledge.
Week 4 gives behavior.

Why simple agents are not enough

Naive tool-using agent

- can choose tools
- can produce fluent answers
- but may skip required knowledge
- may hallucinate missing policy details
- may return pretty output with broken state

Agentic RAG system

- planner decides what knowledge is needed
- retriever grounds the answer in a PDF
- decision layer reasons over retrieved context
- final output is valid JSON + customer reply

A good final answer is not enough.
A reliable AI system needs planning + retrieval + structure.

Agentic RAG Architecture

This is the full system loop students should leave with.

The planner decides what knowledge is needed. Retrieval grounds the answer. The decision layer returns structured output.

Real document source: policy PDF

AI Systems Lab
ThinkPythonAI
thinkpythonai.com

ThinkPythonAI Demo Company Policies

Reference policy PDF for Agentic RAG retrieval and decision demos

Demo Purpose
Use this document as the knowledge source for retrieval, source attribution, and structured AI policy decisions.

1. **Refund Policy**
   Refunds are allowed within 30 days with receipt.
2. **Damage Policy**
   Damaged items can be replaced or issued store credit.
3. **Digital Product Policy**
   Digital products are not refundable once accessed.
4. **Shipping Policy**
   Shipping usually takes 5-7 business days.
5. **Escalation Policy**
   Complex or ambiguous cases must be escalated for manual review.

Suggested Demo Query
"A customer wants a refund after 2 weeks and says the item was damaged. What should we do?"

ThinkPythonAI • Demo Policy Knowledge Base thinkpythonai.com

Why the PDF matters

- Refund Policy - Refunds are allowed within 30 days with receipt.
- Damage Policy - Damaged items can be replaced or issued store credit.
- Digital Product Policy - Digital products are not refundable once accessed.
- Shipping Policy - Shipping usually takes 5-7 business days.
- Escalation Policy - Complex or ambiguous cases must be escalated for manual review.

Planner JSON: the LLM decides what knowledge is needed

Planner role

The first model call does not answer the customer. It classifies the issue, determines the required knowledge, and returns a machine-readable plan.

The model is planning, not answering.

Example planner output

```
{
  "issue_type": "refund_damage_case",
  "knowledge_needed": [
    "refund policy",
    "damage policy"
  ],
  "reason": "Need eligibility + action"
}
```

Code Demo

create_plan() — planner model → JSON

```
def create_plan(user_request):
    prompt = f"""
You are an AI support operations planner.
Analyze the customer request and return ONLY valid JSON.

Customer request:
{user_request}

Return JSON with exactly this schema:
{
  "issue_type": "refund_case | damage_case | refund_damage_case | general_case",
  "knowledge_needed": ["string", "string"],
  "reason": "string"
}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0
    )

    cleaned = clean_json_output(response.choices[0].message.content)
    return json.loads(cleaned)
```

Retrieval over the PDF: chunk → embed → search

1. Load PDF

Extract raw text from a real document source.

2. Chunk text

Turn the PDF into smaller searchable units.

3. Embed chunks

Convert each chunk into a meaning vector.

4. Retrieve Top-K

Compare query vector against chunk vectors.

The corpus changed.
The retrieval engine did not.

Week 3 corpus

PDF lecture chunks about RAG and semantic search

Week 4 corpus

Policy PDF chunks used for business decisions

Code Demo

PDF vector store + semantic retrieval

```python
def build_pdf_vector_store(pdf_path, chunk_size=500):
    pdf_text = load_pdf_text(pdf_path)
    chunks = chunk_text(pdf_text, chunk_size=chunk_size)

    vector_store = []
    for i, chunk in enumerate(chunks):
        emb = get_embedding(chunk)
        vector_store.append({
            "chunk_id": i,
            "source": pdf_path.name,
            "text": chunk,
            "embedding": emb
        })
    return vector_store

def retrieve_policies(query, vector_store, k=3):
    query_embedding = get_embedding(query)
    scored_results = []
    for item in vector_store:
        score = cosine_similarity(query_embedding, item["embedding"])
        scored_results.append({
            "source": item["source"],
            "text": item["text"],
            "score": round(score, 4)
        })
    scored_results.sort(key=lambda x: x["score"], reverse=True)
    return scored_results[:k]
```

Decision JSON: the second model reasons over retrieved policy context

Decision role

The second model sees the original request, the planner JSON, and the retrieved policy chunks. It must reason over that context and return a structured decision object.

First model = planner
Second model = decision maker

Example decision object

```
1 {
2   "eligible": true,
3   "recommended_action": "Offer replacement or store credit",
4   "reasoning": "Within refund window and item is damaged.",
5   "sources_used": ["ThinkPythonAI_Demo_Company_Policies.pdf"],
6   "customer_reply": "Thanks for reaching out..."
7 }
```

Code Demo

make_decision() — retrieved context → decision JSON

```
def make_decision(user_request, plan, retrieved_context):
    prompt = f"""
You are an AI support decision agent.
Use ONLY the retrieved policy context below.

Customer request:
{user_request}

Plan:
{json.dumps(plan, indent=2)}

Retrieved policies:
{json.dumps(retrieved_context, indent=2)}

Return ONLY valid JSON with:
{
  "eligible": true or false,
  "recommended_action": "short action",
  "reasoning": "brief reasoning",
  "sources_used": ["source names"],
  "customer_reply": "polite customer-facing message"
}
"""

    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.0
    )
    return json.loads(clean_json_output(response.choices[0].message.content))
```

Full wrapper: one function ties the whole system together

agentic_rag_support() — planner + retriever + decision

```
def agentic_rag_support(user_request, vector_store, k=3):
    plan = create_plan(user_request)

    query = " ".join(plan["knowledge_needed"])
    retrieved = retrieve_policies(query, vector_store, k=k)

    decision = make_decision(user_request, plan, retrieved)

    return {
        "plan": plan,
        "retrieved": retrieved,
        "decision": decision
    }
```

What makes this premium

- planner JSON is operational
- retrieval is semantic, not keyword based
- decision object is machine-readable
- customer reply is grounded in policy text
- one system now uses all prior weeks

This is not just a chat response.
This is a decision system.

Live demo output: what students should see

PLAN

issue_type = refund_damage_case
knowledge_needed = [refund policy, damage policy]
reason = need eligibility + action

RETRIEVED

Top chunk 1: refund policy
Top chunk 2: damage policy
Scores show semantic relevance, not keyword matching.

DECISION

eligible = true
recommended_action = replacement or store credit
sources_used = [policy PDF]
customer_reply = polished support response

A chatbot answers.
An Agentic RAG system plans, retrieves, reasons, and responds.

AHA

JSON gives structure
RAG gives knowledge
Agents give behavior

Week 2

structured outputs
validation
guardrails

Week 3

embeddings
semantic search
Top-K context

Week 4

planner
decision loop
agentic flow

Wrap-up

What changed in this class

- knowledge now comes from a real PDF
- the first LLM creates a JSON plan
- semantic retrieval finds the right policy chunks
- the second LLM returns a decision object
- the system outputs both machine-readable and human-readable results

Strong closing line

A chatbot answers.
An Agentic RAG system investigates, retrieves, reasons, and responds.

This is where AI starts to look like real software.

Week 4 Class 2 = Agentic RAG
Planner + Retrieval + Decision + Structured Output